

03 - zz - Containerlab Primer - EXERCISE SOLUTIONS

Lesson 1 Exercises: Containerlab Primer

Complete these exercises to solidify your containerlab skills.

```
cd lessons/clab/01-containerlab-primer
```

Exercise 1: Deploy and Explore

Objective: Deploy the lab topology and explore the running environment.

Steps

1. Deploy the provided topology:

```
containerlab deploy -t topology/lab.clab.yml
```

```
[*]-[kc-debian13-test00-
Klon]-[/kubecraft/DrewElliot_kubecraft/lessons/clab/01-containerlab-
primer/topology]
↳ clab deploy
17:19:44 INFO Containerlab started version=0.75.0
17:19:44 INFO Parsing & checking topology file=lab.clab.yml
17:19:44 INFO Creating docker network name=clab IPv4 subnet=172.20.20.0/24
IPv6 subnet=3fff:172:20:20::/64 MTU=0
17:19:44 INFO Creating lab directory
path=/kubecraft/DrewElliot_kubecraft/lessons/clab/01-containerlab-
primer/topology/clab-first-lab
17:19:44 INFO Creating container name=srl2
17:19:44 INFO Creating container name=srl1
17:19:44 INFO Running postdeploy actions kind=srl node=srl2
17:19:44 INFO Created link: srl1:e1-1 ──── srl2:e1-1
17:19:44 INFO Running postdeploy actions kind=srl node=srl1
17:20:14 INFO Adding host entries path=/etc/hosts
17:20:14 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-
first-lab.conf
```

Name	Kind/Image	State	IPv4/6
clab-first-lab-srl1	srl	running	172.20.20.2
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::2
clab-first-lab-srl2	srl	running	172.20.20.3
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::3

✓ done

1. Inspect the running lab:

```
containerlab inspect -t topology/lab.clab.yml
```



```

[*]-[kc-debian13-test00-
Klon]-[/kubecraft/DrewElliot_kubecraft/lessons/clab/01-containerlab-
primer/topology]
└─> docker exec -it clab-first-lab-srl1 sr_cli
Using configuration file(s): []
Welcome to the srlinux CLI.
Type 'help' (and press <ENTER>) if you need any help using this.
--{ running }--[ ]--
A:srl1# show version
-----
-----
-----
-----
Hostname                : srl1
Chassis Type             : 7220 IXR-D2L
Part Number              : Sim Part No.
Serial Number            : Sim Serial No.
System HW MAC Address: 1A:F4:00:FF:00:00
OS                        : SR Linux
Software Version         : v24.10.1
Build Number             : 492-gf8858c5836
Architecture             : x86_64
Last Booted              : 2026-05-16T15:19:49.893Z
Total Memory             : 3921898 kB
Free Memory              : 463339 kB
-----
-----
-----
-----
--{ running }--[ ]--
A:srl1#

```

✓ done

1. Run these commands inside SR Linux and record the output:

```

show version
show interface brief
show system information

```

show version

```
[ Running ]--[ ]--
A:srl1# show version
-----
Hostname           : srl1
Chassis Type       : 7220 IXR-D2L
Part Number        : Sim Part No.
Serial Number      : Sim Serial No.
System HW MAC Address: 1A:F4:00:FF:00:00
OS                 : SR Linux
Software Version   : v24.10.1
Build Number       : 492-gf8858c5836
Architecture       : x86_64
Last Booted        : 2026-05-16T15:19:49.893Z
Total Memory       : 3921898 kB
Free Memory        : 444937 kB
-----
--{ running }--[ ]--
```

show interface brief

```
A:srl1# show interface brief
```

Port		Admin State	Oper State	Speed	Type	Description
ethernet-1/1	enable	up	up	25G		
ethernet-1/2	disable	down	down	25G		
ethernet-1/3	disable	down	down	25G		
ethernet-1/4	disable	down	down	25G		
ethernet-1/5	disable	down	down	25G		
ethernet-1/6	disable	down	down	25G		
ethernet-1/7	disable	down	down	25G		
ethernet-1/8	disable	down	down	25G		
ethernet-1/9	disable	down	down	25G		
ethernet-1/10	disable	down	down	25G		

show system information

DOES NOT EXIST!

```
--{ running }--[ ]--
A:srl1# show system information
Parsing error: Unknown token 'information'. Options are ['#', '>', '>>', 'aaa', 'application', 'lldp', 'logging', 'network-instance', 'sflow', '{', '|']
--{ running }--[ ]--
A:srl1# show system
```

```
A:srl1# show system
```

```
usage: show
```

```
Show the show report of the current context
```

```
Local commands:
```

```
..           Moves up one level
```

```
/           Moves you to the root
```

```
aaa
```

```
application  Show managed applications
```

```
lldp
```

```
logging      Show logging information
```

```
network-instance
```

```
sflow        Show sFlow Data
```

✓ done

2. Exit and connect to `srl2`, verify it's also running:

```
docker exec -it clab-first-lab-srl2 sr_cli
```

Deliverables

Create a file `exercises/exercise1-output.md` with:

- Output of `containerlab inspect`
- SR Linux version from `show version`
- List of interfaces from `show interface brief`
- Your explanation: why is `ethernet-1/1` showing `oper: up` while other ethernet interfaces show `oper: down`?

Exercise 2: Modify the Topology

Objective: Add a third node to the topology.

Steps

1. First, destroy the current lab:

```
containerlab destroy -t topology/lab.clab.yml
```

```
[*]--[kc-debian13-test00-  
Klon]--[kubecraft/DrewElliot_kubecraft/lessons/clab/01-containerlab-  
primer/topology]  
└─> clab destroy -c  
17:37:18 INFO Parsing & checking topology file=lab.clab.yml  
17:37:18 INFO Parsing & checking topology file=lab.clab.yml  
17:37:18 INFO Destroying lab name=first-lab  
17:37:18 INFO Removed container name=clab-first-lab-srl2  
17:37:18 INFO Removed container name=clab-first-lab-srl1  
17:37:18 INFO Removing host entries path=/etc/hosts  
17:37:18 INFO Removing SSH config path=/etc/ssh/ssh_config.d/clab-first-  
lab.conf
```

✓ done

2. Create a new topology file `exercises/three-node.clab.yml` with:

- Three SR Linux nodes: `srl1`, `srl2`, `srl3`
- Linear connectivity: `srl1 -- srl2 -- srl3`
- Use interface `e1-1` for srl1-srl2 link
- Use interface `e1-2` for srl2-srl3 link

three-node.clab.yml

```

# Lesson 1: First Containerlab Topology
# Two SR Linux nodes connected directly
name: ex1threeNode-lab

topology:
  nodes:
    # First SR Linux node
    srl1:
      kind: srl
      image: ghcr.io/nokia/srlinux:24.10.1

    # Second SR Linux node
    srl2:
      kind: srl
      image: ghcr.io/nokia/srlinux:24.10.1

    # Third SR Linux node
    srl3:
      kind: srl
      image: ghcr.io/nokia/srlinux:24.10.1

  links:
    # Connect srl1's ethernet-1/1 to srl2's ethernet-1/1
    - endpoints: ["srl1:e1-1", "srl2:e1-1"]
    - endpoints: ["srl2:e1-2", "srl3:e1-2"]

```

✓ done

2. Deploy your topology:

```
containerlab deploy -t exercises/three-node.clab.yml
```


[*]-[kc-debian13-test00-

Klon]-[/kubecraft/DrewElliot_kubecraft/lessons/clab/01-containerlab-primer/exercises]

└─> clab deploy -t *yaml

17:45:45 INFO Containerlab started version=0.75.0

17:45:45 INFO Parsing & checking topology file=three-node.clab.yaml

17:45:45 INFO Creating docker network name=clab IPv4 subnet=172.20.20.0/24
IPv6 subnet=3fff:172:20:20::/64 MTU=0

17:45:45 INFO Creating lab directory

path=/kubecraft/DrewElliot_kubecraft/lessons/clab/01-containerlab-primer/exercises/clab-ex1threeNode-lab

17:45:45 INFO Creating container name=srl3

17:45:45 INFO Creating container name=srl1

17:45:45 INFO Running postdeploy actions kind=srl node=srl1

17:45:45 INFO Running postdeploy actions kind=srl node=srl3

17:46:13 INFO Creating container name=srl2

17:46:13 INFO Created link: srl1:e1-1 ──── srl2:e1-1

17:46:13 INFO Created link: srl2:e1-2 ──── srl3:e1-2

17:46:13 INFO Running postdeploy actions kind=srl node=srl2

17:46:35 INFO Adding host entries path=/etc/hosts

17:46:35 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-ex1threeNode-lab.conf

Name	Kind/Image	State	IPv4/6 Address
clab-ex1threeNode-lab-srl1	srl	running	172.20.20.2 ghcr.io/nokia/srlinux:24.10.1 3fff:172:20:20::2
clab-ex1threeNode-lab-srl2	srl	running	172.20.20.4 ghcr.io/nokia/srlinux:24.10.1 3fff:172:20:20::4
clab-ex1threeNode-lab-srl3	srl	running	172.20.20.3 ghcr.io/nokia/srlinux:24.10.1 3fff:172:20:20::3

✓ done

3. Verify all three nodes are running:

```
containerlab inspect -t exercises/three-node.clab.yml
```

```
[*]-[kc-debian13-test00-  
Klon]-[/kubecraft/DrewElliot_kubecraft/lessons/clab/01-containerlab-  
primer/exercises]  
└─> containerlab inspect -t three-node.clab.yml  
17:48:20 INFO Parsing & checking topology file=three-node.clab.yml  


| Name                       | Kind/Image                    | State   |
|----------------------------|-------------------------------|---------|
| clab-ex1threeNode-lab-srl1 | srl                           | running |
| 172.20.20.2                | ghcr.io/nokia/srlinux:24.10.1 |         |
| 3fff:172:20:20::2          |                               |         |
| clab-ex1threeNode-lab-srl2 | srl                           | running |
| 172.20.20.4                | ghcr.io/nokia/srlinux:24.10.1 |         |
| 3fff:172:20:20::4          |                               |         |
| clab-ex1threeNode-lab-srl3 | srl                           | running |
| 172.20.20.3                | ghcr.io/nokia/srlinux:24.10.1 |         |
| 3fff:172:20:20::3          |                               |         |


```

✓ done

4. Connect to `srl2` and verify it has two interfaces connected:

```
docker exec -it clab-<your-lab-name>-srl2 sr_cli  
show interface brief
```

```
[*]-[kc-debian13-test00-
Klon]-[/kubecraft/DrewElliot_kubecraft/lessons/clab/01-containerlab-
primer/exercises]
└─> ssh clab-ex1threeNode-lab-srl2
Warning: Permanently added 'clab-ex1threenode-lab-srl2' (ED25519) to the
list of known hosts.
.....
:                               Welcome to Nokia SR Linux!                               :
:                               Open Network OS for the NetOps era.                       :
:                                                                                           :
:   This is a freely distributed official container image.                             :
:                               Use it - Share it                                         :
:                                                                                           :
: Get started: https://learn.srlinux.dev                                                 :
: Container:   https://go.srlinux.dev/container-image                                   :
: Docs:        https://doc.srlinux.dev/24-10                                           :
: Rel. notes:  https://doc.srlinux.dev/rn24-10-1                                       :
: YANG:        https://yang.srlinux.dev/v24.10.1                                       :
: Discord:     https://go.srlinux.dev/discord                                           :
: Contact:     https://go.srlinux.dev/contact-sales                                    :
:                                                                                           :
:                                                                                           :
Using configuration file(s): ['/home/admin/.srlinuxrc']
Welcome to the srlinux CLI.
Type 'help' (and press <ENTER>) if you need any help using this.
```

```
( running )-1-1-
Nokia show interface brief
```

Port	Admin State	Oper State	Speed	Type	Description
ethernet-1/1	enable	up	25G		
ethernet-1/2	enable	up	25G		
ethernet-1/3	disable	down	25G		
ethernet-1/4	disable	down	25G		
ethernet-1/5	disable	down	25G		

Topology Diagram

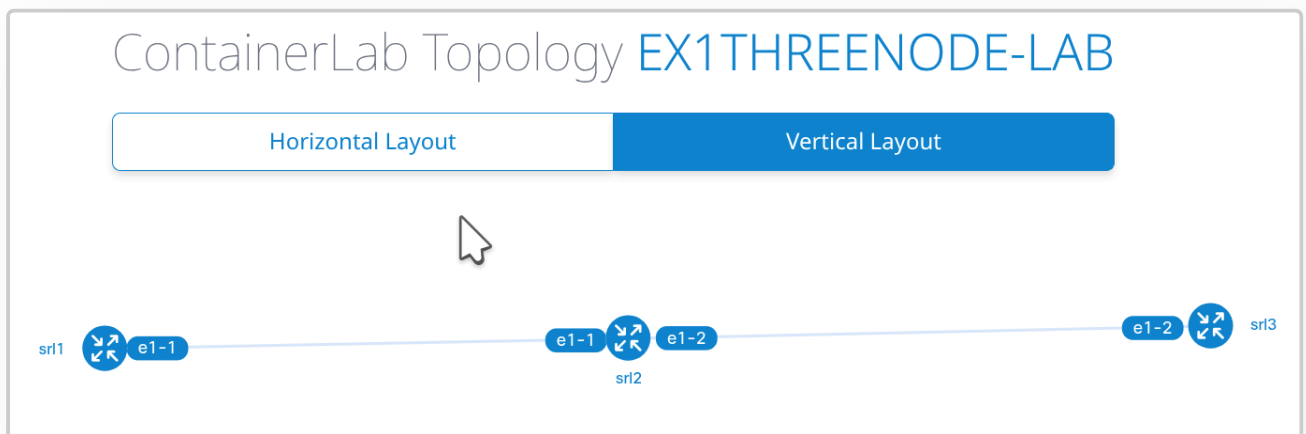
Your topology should look like:



```

[x]-[kc-debian13-test00-
Klon]-[/kubecraft/DrewElliot_kubecraft/lessons/clab/01-containerlab-
primer/exercises]
└─> clab graph -t *.yaml
17:51:07 INFO Parsing & checking topology file=three-node.clab.yaml
17:51:07 INFO Serving topology graph
addresses=
| http://192.168.178.129:50080
| http://172.17.0.1:50080
| http://10.0.0.254:50080
| http://172.20.20.1:50080
| http://[3fff:172:20:20::1]:50080
|
|
kc-debian13-test00-Klon 0- bash 1* bash 2 bash 3 bash

```



Deliverables

- File: `exercises/three-node.clab.yaml`
- Verify: `srl2` shows both `ethernet-1/1` and `ethernet-1/2`

Exercise 3: Generate Documentation

Objective: Generate and save a topology graph.

Steps

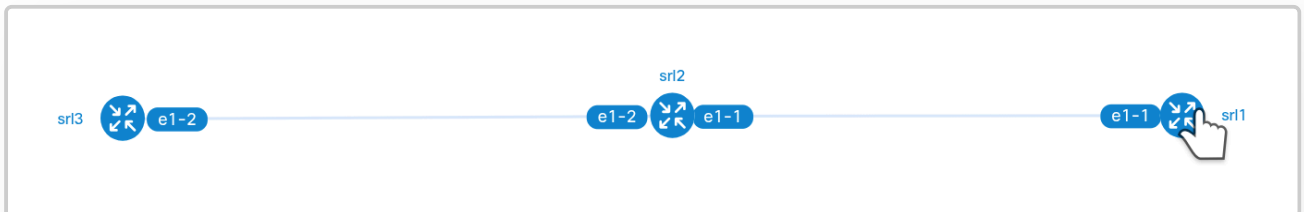
1. With your three-node lab running, generate the graph:

```
containerlab graph -t exercises/three-node.clab.yml -o  
exercises/topology.png
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/exercises]  
└─> clab graph -o three-node.clab.png  
12:59:41 INFO Parsing & checking topology file=three-node.clab.yml  
12:59:41 INFO building graph from topology file  
12:59:41 INFO Serving topology graph  
addresses=  
| http://192.168.178.129:50080  
| http://172.17.0.1:50080  
| http://10.0.0.254:50080
```

It seems this only opens the GUI to watch the dynamic graph

- There is no image (png) written to the specified file



If the above doesn't work (no GUI), try:

```
containerlab graph -t exercises/three-node.clab.yml --drawio
```

2. Take a screenshot or save the output



Also see file at repo

3. Also try the inspect output in different formats:

```
containerlab inspect -t exercises/three-node.clab.yml --format json >
exercises/lab-info.json
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises]
└─> clab inspect -t three-node.clab.yml --format json
13:12:36 INFO Parsing & checking topology file=three-node.clab.yml
...
```

```
{
  "ex1threeNode-lab": [
    {
      "lab_name": "ex1threeNode-lab",
      "labPath": "three-node.clab.yml",
      "absLabPath": "/kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises/three-
node.clab.yml",
      "name": "clab-ex1threeNode-lab-srl1",
      "container_id": "10af6a952ea2",
      "image": "ghcr.io/nokia/srlinux:24.10.1",
      "kind": "srl",
      "state": "running",
      "status": "Up 41 seconds",
      "ipv4_address": "172.20.20.2/24",
      "ipv6_address": "3fff:172:20:20::2/64",
      "owner": "root"
    },
    {
      "lab_name": "ex1threeNode-lab",
      "labPath": "three-node.clab.yml",
      "absLabPath": "/kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises/three-
node.clab.yml",
      "name": "clab-ex1threeNode-lab-srl2",
      "container_id": "95ee503b49f5",
      "image": "ghcr.io/nokia/srlinux:24.10.1",
      "kind": "srl",
      "state": "running",
      "status": "Up 20 seconds",
      "ipv4_address": "172.20.20.4/24",
      "ipv6_address": "3fff:172:20:20::4/64",
      "owner": "root"
    },
    {
      "lab_name": "ex1threeNode-lab",
      "labPath": "three-node.clab.yml",
      "absLabPath": "/kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises/three-
node.clab.yml",
      "name": "clab-ex1threeNode-lab-srl3",
      "container_id": "b358f65cb97e",
      "image": "ghcr.io/nokia/srlinux:24.10.1",
      "kind": "srl",
      "state": "running",
      "status": "Up 41 seconds",
      "ipv4_address": "172.20.20.3/24",
```

```
"ipv6_address": "3fff:172:20:20::3/64",  
  "owner": "root"  
}  
]  
}
```

```
[x]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/exercises]
```

```
└─> clab inspect -t three-node.clab.yml --format table
```

```
13:13:36 INFO Parsing & checking topology file=three-node.clab.yml
```

Name	Kind/Image	State	IPv4/6 Address
clab-ex1threeNode-lab-srl1	srl	running	172.20.20.2
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::2
clab-ex1threeNode-lab-srl2	srl	running	172.20.20.4
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::4
clab-ex1threeNode-lab-srl3	srl	running	172.20.20.3
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::3

Deliverables

- `exercises/topology.png` or screenshot
 - `exercises/lab-info.json`
-

Exercise 4: Find Resources

Objective: Navigate containerlab documentation and community.

Steps

1. **Documentation:** Find the containerlab documentation page for SR Linux and note:

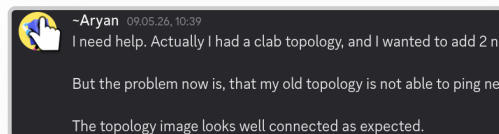
- What startup configuration options are available?
 - **startup-config**: It is possible to provide the startup configuration that the node applies on boot for most Containerlab kinds. The startup config can be provided as:
 1. A path to a file that is available on the host machine and contains the config blob that the node understands.
 2. An embedded config blob that is provided as a multiline string.
 3. An URL (http(s) or S3 (<https://containerlab.dev/manual/s3-usage-example/>)) to a file that contains the config blob that the node can apply.
 - Read more about the usage of the startup configuration (and other ways to perform configuration management with Containerlab) in the Configuration Management (<https://containerlab.dev/manual/config-mgmt/>) section.
- How do you specify a license file (if needed)?
 - **license**: Some containerized NOSes require a license to operate or can leverage a license to lift-off limitations of an unlicensed version. With **license** property a user sets a path to a license file that a node will use. The license file will then be mounted to the container by the path that is defined by the **kind/type** of the node.

2. **GitHub:** Search GitHub for "containerlab" topic and find:

- One repository with an interesting lab topology
 - <https://github.com/ipspace/netlab>: Making virtual networking labs suck less
 - *netlab* (<https://netlab.tools/>) is bringing infrastructure-as-code concepts to networking labs. You'll describe your high-level network topology and routing design in a YAML file, and the tools in this repository will
 - Create *Vagrantfile* configuration file for *libvirt/KVM* environment
 - Create *containerlab* configuration file to run Docker containers
 - Create Ansible inventory and configuration file
 - Create IPv4 and IPv6 addressing plan and OSPFv2, OSPFv3, EIGRP, IS-IS, RIPv2, RIPv6, and BGP routing design
 - Configure IPv4, IPv6, DHCP, DHCPv6, VLANs, VRFs, VXLAN, LLDP, BFD, OSPFv2, OSPFv3, EIGRP, IS-IS, BGP, RIPv2, RIPv6, VRRP, LACP, LAG, MLAG, link bonding, STP, anycast gateways, static routes, route maps, prefix lists, AS-path prefix lists, route redistribution, default route origination, MPLS, BGP-LU, L3VPN (VPNv4 + VPNv6), 6PE, EVPN, SR-MPLS, or SRv6 on your lab devices.
 - Create graphs and reports of your lab topology and BGP, IS-IS, and OSPF routing
 - Configure and manage (virtual) link impairment
 - Provide local- or remote traffic capture capabilities
 - Note the repo URL and what the lab demonstrates
 - Instead of wasting time creating a lab topology in a GUI and configuring tedious details, you'll start with a preconfigured lab that meets your high-level specifications (aka *intent*).

3. **Community:** Visit the containerlab Discord (link in docs) and:

- Note one recent question asked by a user



- I need help. Actually I had a clab topology, and I wanted to add 2 new spines. So I did. Configuration for them I added too. But the problem now is, that my old topology is not able to ping new nodes. And new nodes are able to ping each other(new only). It is like 2 different network inside one, although I have assign same config as others. The topology image looks well connected as expected.
- Find the channel for lab examples
 - **Not found**

Deliverables

Create `exercises/resources.md` with:

- Link to SR Linux documentation page
 - Link to an interesting community lab repo with brief description
 - One observation from the Discord community
-

Exercise 5: Challenge - Add a Linux Host

Objective: Create a topology mixing network OS and Linux containers.

Steps

1. Create `exercises/mixed-topology.clab.yml` with:

- One SR Linux node (`router`)
- One Alpine Linux host (`host1`)
- Connected via router's `e1-1` and host's `eth1`

2. Use this for the Linux node:

```
host1:
  kind: linux
  image: alpine:3.20
```

```
name: mixed-lab
[]
topology:
  nodes:
    router:
      kind: srl
      image: ghcr.io/nokia/srlinux:24.10.1
    host1:
      kind: linux
      image: alpine:3.20
  links:
    - endpoints: ["router:e1-1", "host1:eth1"]
[]
```

3. Deploy and verify both containers are running

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/exercises]  
└─> clab deploy -t mixed-topology.clab.yml  
13:42:42 INFO Containerlab started version=0.75.0  
13:42:42 INFO Parsing & checking topology file=mixed-topology.clab.yml  
13:42:42 INFO Creating docker network name=clab IPv4 subnet=172.20.20.0/24  
IPv6 subnet=3fff:172:20:20::/64 MTU=0  
13:42:42 INFO Creating lab directory  
path=/kubecraft/github_rbeckesch_DrewElliot-kubecraft/lessons/clab/01-  
containerlab-primer/exercises/cla  
b-mixed-lab  
13:42:42 INFO Creating container name=host1  
13:42:42 INFO Creating container name=router  
13:42:42 INFO Running postdeploy actions kind=srl node=router  
13:42:43 INFO Created link: router:e1-1 ──── host1:eth1  
13:42:59 INFO Adding host entries path=/etc/hosts  
13:42:59 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-  
mixed-lab.conf
```

Name	Kind/Image	State	IPv4/6 Address
clab-mixed-lab-host1	linux	running	172.20.20.3
	alpine:3.20		3fff:172:20:20::3
clab-mixed-lab-router	srl	running	172.20.20.2
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::2

```
[x]~[kc-debian13-test00-Klon]~[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/exercises]
```

```
↳ clab inspect -t mixed-topology.clab.yml
```

```
13:43:36 INFO Parsing & checking topology file=mixed-topology.clab.yml
```

Name	Kind/Image	State	IPv4/6 Address
clab-mixed-lab-host1	linux	running	172.20.20.3 3fff:172:20:20::3
clab-mixed-lab-router	srl ghcr.io/nokia/srlinux:24.10.1	running	172.20.20.2 3fff:172:20:20::2

4. Connect to the host:

```
docker exec -it clab-<lab>-host1 sh
```

5. Inside the host, verify the interface exists:

```
ip addr show eth1
```

```
[*]~[kc-debian13-test00-Klon]~[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/exercises]
```

```
↳ docker exec -it clab-mixed-lab-host1 sh
```

```
/ # ip addr show eth1
```

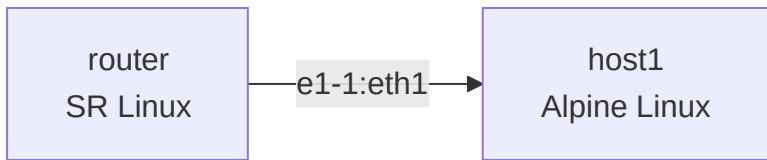
```
141: eth1@if142: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 9500 qdisc  
noqueue state UP
```

```
link/ether aa:c1:ab:5d:b1:96 brd ff:ff:ff:ff:ff:ff
```

```
inet6 fe80::a8c1:abff:fe5d:b196/64 scope link
```

```
valid_lft forever preferred_lft forever
```

Topology



Deliverables

- `exercises/mixed-topology.clab.yml`
 - Verification that `eth1` exists in the Alpine container
-

Exercise 6: Break/Fix -- Container Stopped

Objective: Diagnose why a node is unreachable when its container has stopped.

Setup (break it)

First, make sure the two-node lab is running:

```
cd lessons/clab/01-containerlab-primer
containerlab deploy -t topology/lab.clab.yml
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/topology]
```

```
└─> clab deploy -t lab.clab.yml
```

```
13:46:10 INFO Containerlab started version=0.75.0
```

```
13:46:10 INFO Parsing & checking topology file=lab.clab.yml
```

```
13:46:10 INFO Creating docker network name=clab IPv4 subnet=172.20.20.0/24
```

```
IPv6 subnet=3fff:172:20:20::/64 MTU=0
```

```
13:46:10 INFO Creating lab directory
```

```
path=/kubecraft/github_rbeckesch_DrewElliot-kubecraft/lessons/clab/01-  
containerlab-primer/topology/clab
```

```
-first-lab
```

```
13:46:11 INFO Creating container name=srl1
```

```
13:46:11 INFO Creating container name=srl2
```

```
13:46:11 INFO Running postdeploy actions kind=srl node=srl2
```

```
13:46:11 INFO Created link: srl1:e1-1 ──── srl2:e1-1
```

```
13:46:11 INFO Running postdeploy actions kind=srl node=srl1
```

```
13:46:40 INFO Adding host entries path=/etc/hosts
```

```
13:46:40 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-  
first-lab.conf
```

Name			
Name	Kind/Image	State	IPv4/6
Address			
clab-first-lab-srl1	srl	running	
172.20.20.3			
	ghcr.io/nokia/srlinux:24.10.1		
3fff:172:20:20::3			
clab-first-lab-srl2	srl	running	
172.20.20.2			
	ghcr.io/nokia/srlinux:24.10.1		
3fff:172:20:20::2			

```
[x]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/topology]
```

```
↳ clab inspect -t lab.clab.yml
```

```
13:47:27 INFO Parsing & checking topology file=lab.clab.yml
```

Name	Kind/Image	State	IPv4/6
Address			
clab-first-lab-srl1	srl	running	
172.20.20.3			
	ghcr.io/nokia/srlinux:24.10.1		
3fff:172:20:20::3			
clab-first-lab-srl2	srl	running	
172.20.20.2			
	ghcr.io/nokia/srlinux:24.10.1		
3fff:172:20:20::2			

Now stop one of the containers:

```
docker stop clab-first-lab-srl2
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/topology]
```

```
↳ docker container stop clab-first-lab-srl2
```

```
clab-first-lab-srl2
```

Symptom

```
# Try to connect to srl2
```

```
docker exec -it clab-first-lab-srl2 sr_cli
```

```
# ERROR: container is not running
```

```
# Inspect shows something wrong
```

```
containerlab inspect -t topology/lab.clab.yml
```

```
# srl2 shows state: "exited" instead of "running"
```



```
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/topology]  
└─> docker exec -it clab-first-lab-srl2 sr_cli  
Error response from daemon: container  
7963533a9d2942a72f491caa1e1c52a91fe774dee1346a9721e0f9bc044a93c9 is not  
running
```

```
[x]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/topology]  
└─> containerlab inspect -t lab.clab.yml  
13:49:02 INFO Parsing & checking topology file=lab.clab.yml
```

Name	Kind/Image	State	IPv4/6
clab-first-lab-srl1	srl	running	172.20.20.3
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::3
clab-first-lab-srl2	srl	exited	N/A
	ghcr.io/nokia/srlinux:24.10.1		N/A

Your Task

1. **Diagnose:** Run `containerlab inspect` and identify which node is down. Check `docker ps -a --filter name=clab-first-lab` to see stopped containers.
2. **Fix:** Restart the stopped container and verify it returns to a running state.
3. **Verify:** Connect to srl2 with `docker exec` and confirm `show interface brief` shows `ethernet-1/1` as up.

```
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/topology]
└─> docker ps -a --filter name=clab-first-lab
CONTAINER ID   IMAGE                                COMMAND
CREATED        STATUS          PORTS          NAMES
616998de8d15   ghcr.io/nokia/srlinux:24.10.1      "/tini -- fixuid -q ..." 3
minutes ago    Up 3 minutes    clab-fir
st-lab-srl1
7963533a9d29   ghcr.io/nokia/srlinux:24.10.1      "/tini -- fixuid -q ..." 3
minutes ago    Exited (143) About a minute ago    clab-fir
st-lab-srl2
[]
[]
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/topology]
└─> docker container start clab-first-lab-srl2
clab-first-lab-srl2
[]
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/topology]
└─> docker ps -a --filter name=clab-first-lab
CONTAINER ID   IMAGE                                COMMAND
CREATED        STATUS          PORTS          NAMES
616998de8d15   ghcr.io/nokia/srlinux:24.10.1      "/tini -- fixuid -q ..." 4
minutes ago    Up 4 minutes    clab-first-lab-srl1
7963533a9d29   ghcr.io/nokia/srlinux:24.10.1      "/tini -- fixuid -q ..." 4
minutes ago    Up 6 seconds    clab-first-lab-srl2
[]
[]
```

Deliverables

Document:

- The commands you used to diagnose the problem
- How you fixed it (hint: `docker start` vs. redeploying the lab)
- Why `docker ps` alone wouldn't show the stopped container (you need `-a`)
 - *The question ist already the answer...*

`docker container start ...` seems to be easier and faster, bit `clab redeply ...` is the right /correct way. Both ways seemed to work equally.

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/topology]
```

```
└─> clab redeploy -t lab.clab.yml
```

```
14:01:28 INFO Parsing & checking topology file=lab.clab.yml
```

```
14:01:28 INFO Parsing & checking topology file=lab.clab.yml
```

```
14:01:28 INFO Destroying lab name=first-lab
```

```
14:01:28 ERROR container "clab-first-lab-srl2" exited; container output:
```

```
Sun May 17 11:53:08 UTC 2026: entrypoint.sh called
```

```
Sun May 17 11:53:08 UTC 2026: Calling boot_run script
```

```
net.ipv6.conf.mgmt0.disable_ipv6 = 0
```

```
Actual changes:
```

```
tx-checksum-ip-generic: off
```

```
tx-tcp-segmentation: off [not requested]
```

```
tx-tcp-ecn-segmentation: off [not requested]
```

```
tx-tcp-mangleid-segmentation: off [not requested]
```

```
tx-tcp6-segmentation: off [not requested]
```

```
tx-udp-segmentation: off [not requested]
```

```
tx-checksum-sctp: off
```

```
rx-checksum: off
```

```
1000
```

```
Sun May 17 11:53:11 UTC 2026: entrypoint.sh done, executing sudo -E bash -c
```

```
touch /.dockerenv && /opt/srlinux/bin/sr_linux
```

```
/usr/bin/sudo --non-interactive -- /opt/srlinux/bin/sr_linux --sudo-switched
```

```
sr_app_mgr: Not starting in a named namespace, giving it the name "srbase"
```

```
sr_app_mgr: Unix domain socket directory is /opt/srlinux/var/run/
```

```
Log directory is /var/log/srlinux/stdout
```

```
Found supportd: PID 1295
```

```
Application supportd is running: PID 1295
```

```
Started dev_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;
```

```
/usr/bin/bash -lc "./sr_device_mgr" >>/var/log/srlinux/stdout/dev_mgr.log
```

```
2>&1 &
```

```
Application dev_mgr is running: PID 1727
```

```
Started idb_server: source /etc/profile.d/sr_app_env.sh &>/dev/null;
```

```
/usr/bin/bash -lc "./sr_idb_server" >>/var/log/srlinux/stdout/idb_server.log
```

```
2>&1 &
```

```
Application idb_server is running: PID 1777
```

```
Started aaa_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;
```

```
/usr/bin/bash -lc "./sr_aaa_mgr" >>/var/log/srlinux/stdout/aaa_mgr.log 2>&1
```

```
&
```

```
Started acl_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;
```

```
/usr/bin/bash -lc "./sr_acl_mgr" >>/var/log/srlinux/stdout/acl_mgr.log 2>&1
```

```
&
```

```
Started arp_nd_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;
```

```
/usr/bin/bash -lc "./sr_arp_nd_mgr" >>/var/log/srlinux/stdout/arp_nd_mgr.log
```

```
2>&1 &
```

```
Started bfd_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_bfd_mgr" >>/var/log/srlinux/stdout/bfd_mgr.log 2>&1  
&  
Started chassis_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_chassis_mgr"  
>>/var/log/srlinux/stdout/chassis_mgr.log 2>&1 &  
Started dhcp_client_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_dhcp_client_mgr"  
>>/var/log/srlinux/stdout/dhcp_client_mgr.log 2>&1 &  
Started evpn_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_evpn_mgr" >>/var/log/srlinux/stdout/evpn_mgr.log  
2>&1 &  
Started fib_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_fib_mgr" >>/var/log/srlinux/stdout/fib_mgr.log 2>&1  
&  
Started l2_mac_learn_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_l2_mac_learn_mgr"  
>>/var/log/srlinux/stdout/l2_mac_learn_mgr.log 2>&1 &  
Started l2_mac_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_l2_mac_mgr" >>/var/log/srlinux/stdout/l2_mac_mgr.log  
2>&1 &  
Started lag_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_lag_mgr" >>/var/log/srlinux/stdout/lag_mgr.log 2>&1  
&  
Started license_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_license_mgr"  
>>/var/log/srlinux/stdout/license_mgr.log 2>&1 &  
Started linux_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_linux_mgr" >>/var/log/srlinux/stdout/linux_mgr.log  
2>&1 &  
Started log_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_log_mgr" >>/var/log/srlinux/stdout/log_mgr.log 2>&1  
&  
Started mcid_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_mcid_mgr" >>/var/log/srlinux/stdout/mcid_mgr.log  
2>&1 &  
Started mfib_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_mfib_mgr" >>/var/log/srlinux/stdout/mfib_mgr.log  
2>&1 &  
Started mgmt_server: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_mgmt_server"  
>>/var/log/srlinux/stdout/mgmt_server.log 2>&1 &  
Started net_inst_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_net_inst_mgr"  
>>/var/log/srlinux/stdout/net_inst_mgr.log 2>&1 &  
Started radius_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;  
/usr/bin/bash -lc "./sr_radius_mgr" >>/var/log/srlinux/stdout/radius_mgr.log  
2>&1 &
```

```
Started sdk_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;
/usr/bin/bash -lc "./sr_sdk_mgr" >>/var/log/srlinux/stdout/sdk_mgr.log 2>&1
&
Started sflow_sample_mgr: source /etc/profile.d/sr_app_env.sh &>/dev/null;
/usr/bin/bash -lc "./sr_sflow_sample_mgr"
>>/var/log/srlinux/stdout/sflow_sample_mgr.log 2>&1 &
Started xdp_lc_1: source /etc/profile.d/sr_app_env.sh &>/dev/null;
/usr/bin/bash -lc "exec -a sr_xdp_lc_1 ./sr_xdp_lc --slot_num 1 --
complex_num 2" >>/var/log/srlinux/stdout/xdp_lc_1.log 2>&1 &
Application aaa_mgr is running: PID 1828
Application acl_mgr is running: PID 1864
Application arp_nd_mgr is running: PID 1887
Application bfd_mgr is running: PID 1931
Application chassis_mgr is running: PID 1963
Application dhcp_client_mgr is running: PID 1989
Application evpn_mgr is running: PID 2023
Application fib_mgr is running: PID 2061
Application l2_mac_learn_mgr is running: PID 2098
Application l2_mac_mgr is running: PID 2148
Application lag_mgr is running: PID 2201
Application license_mgr is running: PID 2246
Application linux_mgr is running: PID 2299
Application log_mgr is running: PID 2367
Application mcid_mgr is running: PID 2399
Application mfib_mgr is running: PID 2440
Application mgmt_server is running: PID 2479
Application net_inst_mgr is running: PID 2517
Application radius_mgr is running: PID 2555
Application sdk_mgr is running: PID 2596
14:01:28 INFO Removed container name=clab-first-lab-srl2
14:01:28 INFO Removed container name=clab-first-lab-srl1
14:01:28 INFO Removing host entries path=/etc/hosts
14:01:28 INFO Removing SSH config path=/etc/ssh/ssh_config.d/clab-first-
lab.conf
14:01:28 INFO Containerlab started version=0.75.0
14:01:28 INFO Parsing & checking topology file=lab.clab.yml
14:01:28 INFO Creating docker network name=clab IPv4 subnet=172.20.20.0/24
IPv6 subnet=3fff:172:20:20::/64 MTU=0
14:01:28 INFO Creating lab directory
path=/kubecraft/github_rbeckesch_DrewElliot-kubecraft/lessons/clab/01-
containerlab-primer/topology/clab-first-lab
14:01:28 INFO Creating container name=srl2
14:01:28 INFO Creating container name=srl1
14:01:29 INFO Running postdeploy actions kind=srl node=srl1
14:01:29 INFO Created link: srl1:e1-1 ──── srl2:e1-1
14:01:29 INFO Running postdeploy actions kind=srl node=srl2
14:01:49 INFO Adding host entries path=/etc/hosts
14:01:49 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-
```

first-lab.conf

Name	Kind/Image	State	IPv4/6
clab-first-lab-srl1	srl	running	172.20.20.2
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::2
clab-first-lab-srl2	srl	running	172.20.20.3
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::3

Exercise 7: Break/Fix -- Missing Link

Objective: Diagnose why two nodes cannot see each other when a link is missing from the topology.

Setup (break it)

1. Destroy the current lab:

```
containerlab destroy -t topology/lab.clab.yml --cleanup
```

2. Create a broken topology file `exercises/broken-topology.clab.yml`:

```
name: broken-lab
|
topology:
  nodes:
    srl1:
      kind: srl
      image: ghcr.io/nokia/srlinux:24.10.1
    |
    srl2:
      kind: srl
      image: ghcr.io/nokia/srlinux:24.10.1
    |
  links: []
```

3. Deploy the broken topology:

```
containerlab deploy -t exercises/broken-topology.clab.yml
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises]
└─> clab deploy -t broken-topology.clab.yml
14:05:35 INFO Containerlab started version=0.75.0
14:05:35 INFO Parsing & checking topology file=broken-topology.clab.yml
14:05:35 INFO Creating docker network name=clab IPv4 subnet=172.20.20.0/24
IPv6 subnet=3fff:172:20:20::/64 MTU=0
14:05:35 INFO Creating lab directory
path=/kubecraft/github_rbeckesch_DrewElliot-kubecraft/lessons/clab/01-
containerlab-primer/exercises/clab-broken-lab
14:05:35 INFO Creating container name=srl1
14:05:35 INFO Creating container name=srl2
14:05:35 INFO Running postdeploy actions kind=srl node=srl1
14:05:35 INFO Running postdeploy actions kind=srl node=srl2
14:06:04 INFO Adding host entries path=/etc/hosts
14:06:04 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-
broken-lab.conf
```

Name	Kind/Image	State	IPv4/6
clab-broken-lab-srl1	srl	running	172.20.20.2
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::2
clab-broken-lab-srl2	srl	running	172.20.20.3
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::3

Symptom

Both containers are running, but:

```
docker exec -it clab-broken-lab-srl1 sr_cli -c "show interface brief"
# ethernet-1/1 shows "down" (no link partner)
```


Your Task

1. **Diagnose:** Both nodes are running, so it's not a container problem. Check `show interface brief` on both nodes -- what's different from the working lab?
 - The output shows that `oper` is `down`, not `up`
2. **Explain:** Why does `ethernet-1/1` show `oper: down`? What's missing?
 - The links are missing. If they existed, the `oper` would be `up` because they were connected
3. **Fix:** Destroy the broken lab, add the missing link to the topology file, and redeploy.
4. **Verify:** Confirm `ethernet-1/1` is now `oper: up` on both nodes.

```
[x]~[kc-debian13-test00-Klon]~[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer/exercises]  
└─> clab destroy -t broken-topology.clab.yml  
14:10:06 INFO Parsing & checking topology file=broken-topology.clab.yml  
14:10:07 INFO Parsing & checking topology file=broken-topology.clab.yml  
14:10:07 INFO Destroying lab name=broken-lab  
14:10:07 INFO Removed container name=clab-broken-lab-srl1  
14:10:07 INFO Removed container name=clab-broken-lab-srl2  
14:10:07 INFO Removing host entries path=/etc/hosts  
14:10:07 INFO Removing SSH config path=/etc/ssh/ssh_config.d/clab-broken-  
lab.conf
```

```
- endpoints: ["srl1:e1-1", "srl2:e1-1"]
```

broken-topology-repaired.clab.yml

```
name: broken-lab  
  
topology:  
  nodes:  
    srl1:  
      kind: srl  
      image: ghcr.io/nokia/srlinux:24.10.1  
    srl2:  
      kind: srl  
      image: ghcr.io/nokia/srlinux:24.10.1  
  links:  
    - endpoints: ["srl1:e1-1", "srl2:e1-1"]
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises]
└─> clab deploy -t broken-topology-repaired.clab.yml
14:12:45 INFO Containerlab started version=0.75.0
14:12:45 INFO Parsing & checking topology file=broken-topology-
repaired.clab.yml
14:12:45 INFO Creating docker network name=clab IPv4 subnet=172.20.20.0/24
IPv6 subnet=3fff:172:20:20::/64 MTU=0
14:12:45 INFO Creating lab directory
path=/kubecraft/github_rbeckesch_DrewElliot-kubecraft/lessons/clab/01-
containerlab-primer/exercises/clab-broken-lab
14:12:45 INFO Creating container name=srl1
14:12:45 INFO Creating container name=srl2
14:12:46 INFO Running postdeploy actions kind=srl node=srl2
14:12:46 INFO Created link: srl1:e1-1 ----- srl2:e1-1
14:12:46 INFO Running postdeploy actions kind=srl node=srl1
14:13:06 INFO Adding host entries path=/etc/hosts
14:13:07 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-
broken-lab.conf
```

Name	Kind/Image	State	IPv4/6
clab-broken-lab-srl1	srl	running	172.20.20.3
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::3
clab-broken-lab-srl2	srl	running	172.20.20.2
	ghcr.io/nokia/srlinux:24.10.1		3fff:172:20:20::2

Warning

It is not enough to `destroy` and `deploy`, or to `redploy`. You have to `destroy -c` / `destroy --clean` or the network devices will NOT come up!

Deliverables

- Your fixed `exercises/broken-topology.clab.yml`
 - Explanation of the difference between a node being "running" and an interface being "up"
-

Cleanup

After completing all exercises:

```
# Destroy any running labs
containerlab destroy --all
# Verify
docker ps | grep clab
```

```
[x]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises]
└─> clab destroy --all
14:19:09 WARN The following labs will be removed:
labs=
| 1. Topology: /kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises/broken-topology-
repaired.clab.yml
| Lab Dir: /kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises/clab-broken-lab
|
Are you sure you want to remove all labs listed above? Enter 'y', to confirm
or ENTER to abort: y
14:19:23 INFO Parsing & checking topology file=broken-topology-
repaired.clab.yml
14:19:23 INFO Destroying lab name=broken-lab
14:19:23 INFO Removed container name=clab-broken-lab-srl1
14:19:23 INFO Removed container name=clab-broken-lab-srl2
14:19:23 INFO Removing host entries path=/etc/hosts
14:19:23 INFO Removing SSH config path=/etc/ssh/ssh_config.d/clab-broken-
lab.conf
|
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/01-containerlab-primer/exercises]
└─> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
f4bcc1cf4c98	hello-world	"/hello"	39 hours ago	Exited (0) 39 hours ago
	eager_tu			

Validation

Run the automated tests:

```
pytest tests/ -v
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/01-containerlab-primer]
```

```
└─> pytest tests/ -v
```

```
=====
```

```
===== test session starts
```

```
=====
```

```
=====
```

```
platform linux -- Python 3.13.5, pytest-8.3.5, pluggy-1.5.0 --
```

```
/usr/bin/python3
```

```
cachedir: .pytest_cache
```

```
rootdir: /kubecraft/github_rbeckesch_DrewElliot-kubecraft
```

```
configfile: pyproject.toml
```

```
plugins: anyio-4.8.0
```

```
collected 18 items
```

```
tests/test_lab.py::TestEnvironment::test_containerlab_installed PASSED
```

```
[ 5%]
```

```
tests/test_lab.py::TestEnvironment::test_docker_available PASSED
```

```
[ 11%]
```

```
tests/test_lab.py::TestEnvironment::test_topology_file_exists PASSED
```

```
[ 16%]
```

```
tests/test_lab.py::TestEnvironment::test_topology_file_valid_yaml PASSED
```

```
[ 22%]
```

```
tests/test_lab.py::TestTopologyStructure::test_has_name PASSED
```

```
[ 27%]
```

```
tests/test_lab.py::TestTopologyStructure::test_has_nodes PASSED
```

```
[ 33%]
```

```
tests/test_lab.py::TestTopologyStructure::test_nodes_have_kind PASSED
```

```
[ 38%]
```

```
tests/test_lab.py::TestTopologyStructure::test_nodes_have_image PASSED
```

```
[ 44%]
```

```
tests/test_lab.py::TestTopologyStructure::test_has_links PASSED
```

```
[ 50%]
```

```
tests/test_lab.py::TestTopologyStructure::test_links_have_endpoints PASSED
```

```
[ 55%]
```

```
tests/test_lab.py::TestTopologyStructure::test_no_latest_tags PASSED
```

```
[ 61%]
```

```
tests/test_lab.py::TestLabDeployment::test_lab_deploys_successfully PASSED
```

```
[ 66%]
```

```
tests/test_lab.py::TestLabDeployment::test_containers_running_after_deploy
```

```
PASSED
```

```
[ 72%]
```

```
tests/test_lab.py::TestLabDeployment::test_inspect_returns_data PASSED
```

```
[ 77%]
```

```
tests/test_lab.py::TestLabDeployment::test_lab_destroys_successfully PASSED
```

```
[ 83%]
```

```
tests/test_lab.py::TestExerciseFiles::test_solutions_directory_exists PASSED
```

```
[ 88%]
tests/test_lab.py::TestExerciseFiles::test_solution_topologies_valid PASSED
[ 94%]
tests/test_lab.py::TestExerciseFiles::test_solution_topologies_no_latest_tag
s PASSED
[100%]
=====
18 passed in 161.02s (0:02:41)
=====
```

Completion Checklist

- ✓ Exercise 1: Deployed and explored the two-node lab
- ✓ Exercise 2: Created and deployed three-node topology
- ✓ Exercise 3: Generated topology graph and JSON output
- ✓ Exercise 4: Found documentation and community resources
- ✓ Exercise 5: Created mixed network/Linux topology
- ✓ Exercise 6: Diagnosed and fixed a stopped container
- ✓ Exercise 7: Diagnosed and fixed a missing link

Next Steps

Commit your exercises to your fork and proceed to Lesson 2: IP Fundamentals.

- ✓ will be done after I exported this Excercise as PDF to the repo